



5주차

📅 Date	@2025/02/13
≡ 내용	NLP 논문리뷰(word2vec, ELMo)
⚡ 세션 진행	완료

Efficient Estimation of Word Representations in Vector Space

1. 연구 배경 및 목표

- 기존의 원핫인코딩 방식은 단어 간 유사성을 반영하지 못하며, 단어 집합이 커질수록 벡터의 차원이 커져 계산 비용이 증가하는 한계점이 있음
- 이를 해결하기 위해 신경망 기반 분산 표현을 활용하여 단어 간 의미적, 구문적 관계를 반영하는 벡터를 학습시키고자 함
- 특히, 대량의 데이터를 활용하면서도 연산량을 최소화할 수 있는 **CBOW, Skip-gram** 제안

2. 기존 연구 및 한계

(1) N-gram 모델

- 특정 단어의 등장 확률을 통계적으로 계산하는 방법
- 희소성 문제 - 데이터가 충분하지 않으면 예측 성능이 저하됨
- n 선택의 트레이드 오프 - n을 크게 하면 성능이 향상되지만 계산량이 급증하는 트레이드 관계를 가짐

(2) NNLM

- 입력 단어들을 벡터로 변환 후 은닉층을 거쳐 softmax로 출력
- 연산량이 많으며 학습 속도가 느림
- 보완 : **Huffman Tree 기반 Hierarchical Softmax**를 사용하여 연산 최적화

(3) RNNLM

- 은닉 상태를 유지하여 문맥을 반영할 수 있음
- NNLM은 input context length를 지정해줘야 한다는 단점을 RNN 구조를 통해 개선한 모델

- 연산량이 커서 비효율적임
- 보완 : **Binary Tree Softmax**로 연산량 감소

3. 제안 모델

1) Continuous Bag-of-Words (CBOW)

- 문맥을 기반으로 현재 단어를 예측하는 방식
- 특징
 - 은닉층이 없어 연산량이 적고 빠름
 - 단어의 순서를 고려하지 않음
 - 주변 단어들을 평균 내어 벡터로 변환하여 출력층으로 전달
- 장점
 - 빠른 속도
 - 높은 학습 효율
- 단점
 - 문맥 순서를 고려하지 않아 일부 의미 정보가 손실될 수 있음

2) Skip-gram

- 현재 단어에서 주변 단어를 예측하는 방식
- 특징
 - 문맥 예측 범위를 조절하여 더 풍부한 의미 학습 가능
 - 현재 단어와 거리가 먼 단어일수록 확률적으로 낮은 가중치를 부여
- 장점
 - 의미적으로 더 풍부한 벡터 학습 가능
- 단점
 - CBOW보다 계산량이 많음

4. 실험 결과

- 학습 데이터의 크기와 임베딩 차원에 따른 정확도 비교 - 학습 데이터의 수와 임베딩 차원의 수를 각각 증가시키는 것보다는 함께 증가시킬 때 더 큰 효과가 있었음

- 모델 간 성능 비교 - CBOW, Skip-gram이 Sementic, Syntactic 모두에서 RNNLM, NNLM보다 좋은 정확도를 보임
- 에폭과 학습 데이터 수에 대한 성능 비교 - 같은 모델에서 비교해보면 3에폭에 비해 10에폭을 2배의 데이터로 돌리는 것이 오히려 학습 시간과 정확도 지표에서 더 좋은 결과를 불러옴

5. 결론

- 기존 NNLM, RNNLM보다 연산량이 적고 학습 속도가 빠르면서도 높은 정확도를 보이는 Word2vec 모델을 제안함
- CBOW는 빠르고 계산 효율적이며 Skip-gram은 더 풍부한 의미적 관계를 학습함
- 대규모 데이터에서 효과적으로 학습 가능하며, 다양한 NLP 응용에 활용될 수 있음

ELMo : Deep contextualized word representations

1. 연구 배경 및 목표

- 기존의 Word Embedding 기법(Word2vec, GloVe 등)은 단어가 각 문장에서 쓰이는 문맥이 다르더라도, 같은 단어는 무조건 한 개의 고정된 벡터로 표현함.
- 따라서, 문법 구조나 다의어에 따른 의미 변형을 포착하기 어렵다는 문제점이 존재
- 이를 해결하기 위해 ELMo를 제안
 - 양방향 LSTM을 활용하여 전체 문장의 문맥적 특성을 각 위치의 단어 임베딩 벡터에 반영
 - 같은 단어라도 서로 다른 의미를 가지면 다른 벡터로 표현함으로써 문제 해결

2. 기존 연구 및 한계

(1) Character Based CNN을 통한 임베딩

- 단어를 철자 단위로 분해하여 CNN을 적용
- 단어 내부 특징을 잘 반영하지만 문맥 정보를 반영하지 못한다는 한계

(2) Word2Vec + Clustering

- 문맥 벡터를 클러스터링하여 서로 다른 문맥에서 다른 임베딩을 갖도록 함
- 하지만 최적의 클러스터 개수를 정하는 것이 어려움

(3) 양방향 LSTM 선행 연구

- 양방향 LSTM 활용은 단일 언어 환경이 아니라 초기 기계번역 TASK에서처럼 소스-타겟 문장 쌍의 병렬 코퍼스를 사용했다는 한계점이 존재

3. ELMo 모델 아키텍처

(1) Character-based CNN (초기 단어 임베딩)

- 단어를 문자 단위로 분해하고 CNN을 적용해 초기 벡터를 생성
- Highway Network를 사용해 중요한 정보만 보존

(2) Bidirectional Language Model (BiLM)

- 양방향 LSTM을 사용해 문맥을 반영한 단어 표현 학습
- **Forward LM:** 현재 토큰을 기준으로 다음에 올 토큰을 예측
- **Backward LM:** 현재 토큰을 기준으로 이전 토큰들을 예측
- 두 방향을 함께 학습하여 문맥 정보를 효과적으로 반영함

(3) ELMo Embedding

- BiLSTM의 여러 층에서 출력된 벡터를 가중합(Weighted Sum)하여 최종 단어 벡터 생성
- 각 층의 중요도는 학습 가능한 가중치로 조정되며, 태스크(Task)별로 최적화됨

(4) Task-Specific Layer

- 기존 모델의 입력 또는 출력에 ELMo 벡터를 추가하여 성능 향상
- 앞단 - 텍스트 분류, 감성분류
- 뒷단 - QA, 요약 등

4. 실험 결과

- 단지 ELMo를 추가하는 것 만으로도 6개의 NLP task에서 당시 소타 달성
- SQuAD, SNLI, SRL 등의 Task에서 성능이 향상됨
- 기존 모델은 486 epoch에서 최고 성능을 달성했다면 ELMo 적용 모델은 10 epoch 만에 동일한 성능 달성
- 훈련 데이터가 적을 때도 성능이 더 뛰어남

5. 결론

- BiLM을 통해 기존 Word 단위가 아닌 Context 영역에서 Embedding을 가능하게 하여 기존 다의어 문제와 고정 임베딩 문제 해결
- 모델 내부 모든 Layer를 활용함으로써, 문법과 맥락 모두 기반한 더욱 정밀한 언어 이해
- 태스크 별 조정을 가능하게 하여, 다양한 Downstream Task에 범용적으로 접근 가능

 Word2vec

 ELMo